Question 1:

Input File	K	M	FIFO	LRU	OPTFF
File 1	4	60	47	49	26
File 2	5	72	54	56	29
File 3	3	80	54	45	33

a) In all 3 outputs shown above, OPTFF has the fewest number of misses out of all the policies, regardless of the number of requests.
b) Overall, FIFO and LRU were consistently close; however one policy was not consistently better than the other. As for File 1 and File 2, FIFO out performed LRU by 2 misses, while in File 3, LRU out performed FIFO by 9 misses. This suggests that these policies work fairly similar on certain workloads, but LRU can perform better due to relying on recency of access rather than arrival order.

Question 2:

For k=3, there does exist a sequence for which OPTFF incurs strictly fewer misses than LRU (or FIFO). For example, the sequence:

3 6

1 2 3 4 2 1

Produces the output:

FIFO : 5
LRU : 5
OPTFF : 4

Therefore, clearly there exists a sequence that OPTFF has strictly fewer misses than both of the other policies. This is because OPTFF uses knowledge of future requests when deciding which page to evict rather than usage recency or arrival time. For example in this case, after the cache is filled with **1,2, and 3,** the request for 4 forces an eviction. Since the remaining requests are **2, 1**, page 3 is never used again which is then evicted by the OPTFF policy and avoids an extra miss. FIFO and LRU are not aware of future requests so they evict a less optimal one and add an additional miss.

Question 3:

To prove that OPTFF is the optimal eviction algorithm, we show a case that the number of misses of OPTFF is no larger than that of (A) on any fixed sequence.

By using induction on the number of steps, we say the base case is where j = 0 which is trivially true, and the invariant that there exists an optimal schedule A that has the same eviction schedule as OPTFF through the first j steps.

Inductive step: Say the invariant holds through j steps, then *r* is the request at step j+1, where A and OPTFF agreed so far through step j, meaning they have the same cache contents. There are a few cases that determine OPTFF is the optimal eviction algorithm.

The first case is that *r* is already in the cache, neither algorithm evicts anything since OPTFF algorithm and A have the same contents and agrees with each other so far.

The second case is that *r* is not in the cache and A, and OPTFF evict the same item, since they already agree up to j+1 steps, the invariant holds.

Then finally case 3, where *r* is not in the cache, and A and OPTFF evict different items. Say OPTFF evicts page *d* and A evicts *f* , where *f* does not equal *d.* Since OPTFF chooses the page who's next use is farthest in the future, *d* will always be at least the same or further than *f*. Therefore replacing A's choice with OPTFF's choice cannot increase the number of future misses. Using this logic, modifying A so it evicts *d* instead allows the new schedule to be optimal and agrees with OPTFF through j+1.

Therefore, by induction, there exists an optimal schedule that agrees with OPTFF on every step, therefore OPTFF is optimal.